

Aula 5 - Herança e Polimorfismo (POO II)

Explorando os pilares fundamentais da Programação Orientada a Objetos: como criar hierarquias de classes eficientes e implementar comportamentos polimórficos em Python.

O que é Herança?

Reutilização de Código

Herança permite criar novas classes baseadas em classes existentes, aproveitando atributos e métodos já implementados.

Relação Pai-Filho

A classe filha herda automaticamente tudo da classe pai, podendo adicionar ou modificar funcionalidades conforme necessário.

Evita Repetição

Elimina código duplicado ao centralizar funcionalidades comuns na classe base, facilitando manutenção.

Sintaxe Básica em Python

```
class Pai:
    def falar(self):
        print("Olá, eu sou o pai!")

class Filho(Pai):
    pass

f = Filho()
f.falar() # Herda o método da classe Pai
```

Exemplo Prático: Sistema de Pessoas

```
class Pessoa:
    def __init__(self, nome):
        self.nome = nome

    def apresentar(self):
        print(f"Olá, meu nome é {self.nome}")

class Aluno(Pessoa):
    def estudar(self):
        print(f"{self.nome} está estudando.")
```

```
p1 = Aluno("Carlos")
p1.apresentar()
# Saída: Olá, meu nome é Carlos

p1.estudar()
# Saída: Carlos está estudando.
```

A classe Aluno herda o método apresentar() e adiciona o método estudar()

Exercício 1: Praticando Herança

1

Crie a Classe Base

Implemente uma classe chamada **Animal** com um método `emitir_som()` que será herdado pelas classes filhas.

2

Classe Cachorro

Crie a classe **Cachorro** que herda de **Animal**. O método deve imprimir "Au au!" quando chamado.

3

Classe Gato

Crie a classe **Gato** que também herda de **Animal**. O método deve imprimir "Miau!" quando executado.



Dica importante: Use a sintaxe `class Cachorro(Animal):` para estabelecer a relação de herança entre as classes.

Este exercício demonstra como diferentes classes podem herdar a mesma estrutura base, mantendo suas particularidades individuais.



Sobrescrita de Métodos



Classe Pai

Define o comportamento padrão que pode ser herdado



Sobrescrita

Classe filha redefine o método com novo comportamento



Novo Comportamento

Método executado com a lógica personalizada

Uma classe filha pode **sobrescrever** métodos herdados da classe pai, alterando seu comportamento para atender necessidades específicas. Isso permite customização mantendo a estrutura de herança.

Exemplo de Sobrescrita

```
class Pessoa:
    def falar(self):
        print("Olá!")

class Professor(Pessoa):
    def falar(self):
        print("Olá, turma!")

p1 = Professor()
p1.falar() # Saída: Olá, turma!
```

Neste exemplo, o método `falar()` foi **redefinido** na classe Professor, substituindo completamente o comportamento original da classe Pessoa.

O que é Polimorfismo?



Muitas Formas

Polimorfismo vem do grego e significa "muitas formas" — um conceito fundamental em POO.



Mesmo Método

Objetos de diferentes classes podem usar métodos com o mesmo nome.



Comportamentos Diferentes

Cada classe implementa o método de forma única, adequada ao seu contexto.

Exemplo Prático: Cálculo de Áreas

```
class Forma:
    def area(self):
        pass

class Quadrado(Forma):
    def area(self):
        return 4 * 4

class Circulo(Forma):
    def area(self):
        return 3.14 * (3 ** 2)

formas = [Quadrado(), Circulo()]
for f in formas:
    print(f.area())
```

Resultado da Execução

Saída:

- 16 (área do quadrado)
- 28.26 (área do círculo)

Mesmo método `area()`, mas cada classe calcula de forma diferente!





Exercício 2: Polimorfismo em Ação

01

Classe Base Veiculo

Crie uma classe `Veiculo` com um método `mover()` que servirá como base para as classes filhas.

03

Implemente Bicicleta

Crie a classe `Bicicleta` também herdando de `Veiculo`. O método deve imprimir "A bicicleta está se movendo".

02

Implemente Carro

Crie a classe `Carro` herdando de `Veiculo`. Sobrescreva o método para imprimir "O carro está se movendo".

04

Teste o Polimorfismo

Crie uma lista contendo objetos de ambas as classes e use um loop `for` para chamar `mover()` em cada um.

Este exercício demonstra o poder do polimorfismo: podemos tratar diferentes tipos de veículos de forma uniforme, mesmo que cada um se comporte de maneira única.

Usando super()

A função `super()` é uma ferramenta poderosa que permite chamar **atributos da classe pai**.

É útil quando queremos **adicionar** características sem perder o que já existe.

```
class Pessoa:
    def __init__(self, nome):
        self.nome = nome

class Aluno(Pessoa):
    def __init__(self, nome, matricula):
        super().__init__(nome)
        self.matricula = matricula
```




Exercício 3: Praticando super()

1

Crie a Classe Funcionario

Implemente uma classe base `Funcionario` com os atributos `nome` e `salario` no construtor.

2

Crie a Classe Gerente

Implemente a classe `Gerente` que herda de `Funcionario` e adiciona o atributo `departamento`.

3

Use super()

No construtor de `Gerente`, use `super().__init__(nome, salario)` para reaproveitar o construtor da classe pai.

4

Teste Sua Implementação

Crie um objeto `Gerente` e imprima todos os seus atributos para verificar se a herança está funcionando corretamente.

Este exercício reforça o conceito de extensão de classes: aproveitamos o que já existe e adicionamos apenas o que é necessário, seguindo o princípio DRY (Don't Repeat Yourself).

Herança Múltipla

Em Python, uma classe pode **herdar de várias classes simultaneamente**, um recurso chamado **herança múltipla**. Isso permite combinar funcionalidades de diferentes classes base.

Animal
Características básicas de animais



Voador
Capacidade de voar



Corredor
Locomoção terrestre



Aquático
Viver na água



Exemplo de Herança Múltipla

```
class Animal:
    def comer(self):
        print("Comendo...")

class Voador:
    def voar(self):
        print("Voando...")

class Passaro(Animal, Voador):
    pass

p = Passaro()
p.comer() # Herdado de Animal
p.voar()  # Herdado de Voador
```

📌 **Atenção:** A herança múltipla deve ser usada **com cuidado**, pois pode gerar **conflitos de métodos** quando duas classes pai têm métodos com o mesmo nome. Python resolve isso usando o MRO (Method Resolution Order).

Conclusão: Dominando POO

Herança

Reaproveitamento de código através de hierarquias de classes, facilitando manutenção e organização.

Herança Múltipla

Permite combinar funcionalidades de várias classes, expandindo possibilidades de design.

Polimorfismo

Métodos com mesmo nome, comportamentos diferentes — flexibilidade e código elegante.

super()

Mantém o vínculo entre classes pai e filha, evitando duplicação de código.

Próximos Passos

Na próxima aula, avançaremos com **encapsulamento e classes abstratas**, aprendendo a proteger dados e criar interfaces mais robustas. Esses conceitos tornarão seus programas ainda mais organizados, seguros e profissionais.

"A herança e o polimorfismo são ferramentas que transformam código repetitivo em arquiteturas elegantes e reutilizáveis."